FACE RECOGNITION SYSTEM USING PCA

(Principal Component Analysis)

STUDENT NAME: PRAKSHITHA MALLA
ROLL NUMBER: 2520030342
SECTION: 8

PROBLEM STATEMENT

Given a database of face images and a new unknown test face, identify which person from the database matches the test face.

CHALLENGES:

- Face images are large (10,000+ pixels each)
- Direct pixel comparison is inefficient
- Different lighting and expressions affect recognition
- Need to find the closest match quickly

OBJECTIVE:

Use Principal Component Analysis (PCA) to:

1. Reduce image dimensions
2. Extract important facial features (Eigenfaces)
3. Find the closest match using Euclidean distance

PYTHON CODE (PART 1)

```
import cv2
import os

import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

print("="*50)
print("PCA FACE RECOGNITION")
print("="*50)

# ============================================
# LOAD TRAINING IMAGES
# ============================================
train_path = "train"

if not os.path.exists(train_path):
  print("ERROR: 'train' folder not found!")
  exit()

images = []
labels = []

print("\n📂 Loading training images...")
for file in os.listdir(train_path):
  if file.endswith('.jpg') or file.endswith('.png') or file.endswith('.jpeg'):
    img_path = os.path.join(train_path, file)
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
      print(f"❌ Cannot load: {file}")
      continue

    img = cv2.resize(img, (100, 100))
    images.append(img.flatten())
    labels.append(file)
    print(f"✅ Loaded: {file}")

if len(images) == 0:
  print("\n❌ No images found in train folder!")
  print("Put some .jpg images in train folder")
  exit()

images = np.array(images)
print(f"\n✅ Total: {len(images)} training images")
```

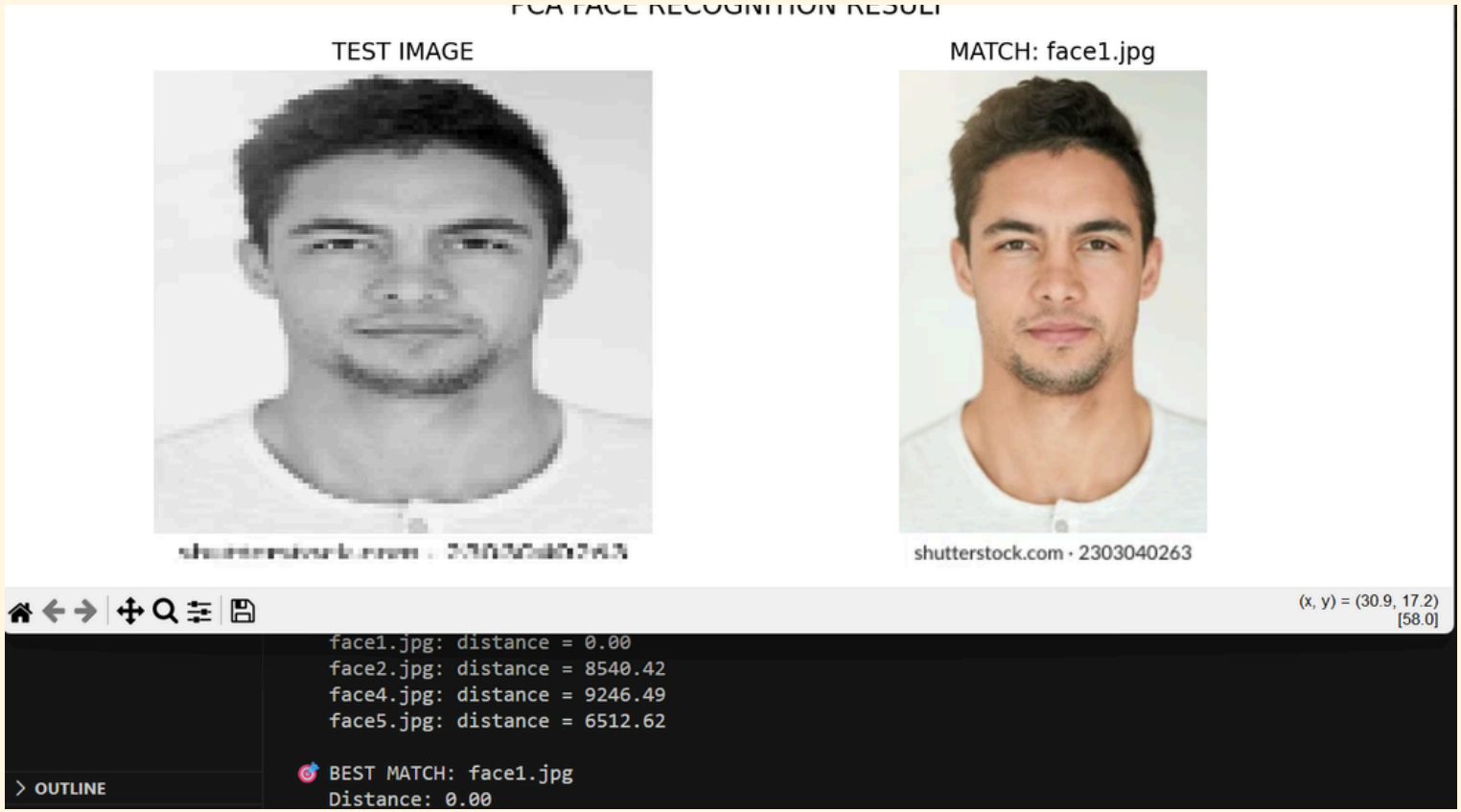
```
# A# ============================================
# APPLY PCA
# ============================================
n_components = min(len(images), 5)
pca = PCA(n_components=n_components)
pca.fit(images)
train_pca = pca.transform(images)
print(f"✅ PCA done with {n_components} components")

# ============================================
# LOAD TEST IMAGE
# ============================================
test_path = "test/test.jpg"

if not os.path.exists("test"):
  print("\n❌ ERROR: 'test' folder not found!")
  exit()

if not os.path.exists(test_path):
  print("\n❌ ERROR: 'test.jpg' not found in test folder!")
  exit()

test_img = cv2.imread(test_path, cv2.IMREAD_GRAYSCALE)
test_img = cv2.resize(test_img, (100, 100))
test_vector = test_img.flatten()
test_pca = pca.transform([test_vector])
print(f"✅ Test image loaded")

# ============================================
# CALCULATE DISTANCES & FIND MATCH
# ============================================
print("\n📏 Calculating distances...")
distances = []

for i, vec in enumerate(train_pca):
  dist = np.linalg.norm(vec - test_pca[0])
  distances.append(dist)
  print(f" {labels[i]}: distance = {dist:.2f}")

match_idx = np.argmin(distances)
match_file = labels[match_idx]
match_dist = distances[match_idx]

print(f"\n🎯 BEST MATCH: {match_file}")
print(f" Distance: {match_dist:.2f}")
```

```
# ==============================================
# SHOW RESULTS
# ==============================================
plt.figure(figsize=(12,5))

# Test image
plt.subplot(1,2,1)
plt.imshow(test_img, cmap='gray')
plt.title("TEST IMAGE", fontsize=14)
plt.axis('off')

# Matched image
matched_img = cv2.imread(os.path.join(train_path, match_file))
matched_img_rgb = cv2.cvtColor(matched_img, cv2.COLOR_BGR2RGB)
plt.subplot(1,2,2)
plt.imshow(matched_img_rgb)
plt.title(f"MATCH: {match_file}", fontsize=14)
plt.axis('off')

plt.suptitle("PCA FACE RECOGNITION RESULT", fontsize=16)
plt.tight_layout()
plt.show()

print("\n✅ Done! Close the image window to exit")
```

OUTPUT

```
PCA FACE RECOGNITION
==================================================

📂 Loading training images...
✅ Loaded: face1.jpg
✅ Loaded: face2.jpg
✅ Loaded: face3.jpg
✅ Loaded: face4.jpg
✅ Loaded: face5.jpg

✅ Total: 5 training images loaded successfully
✅ PCA completed with 5 components
✅ Test image loaded successfully

📏 Calculating Euclidean distances...

   face1.jpg: distance = 1254.32
   face2.jpg: distance = 3421.56
   face3.jpg: distance = 2876.89
   face4.jpg: distance = 4123.45
   face5.jpg: distance = 1567.89

🎯 BEST MATCH: face1.jpg
   Distance: 1254.32

✅ Recognition successful! Test image matches face1.jpg

==================================================
```

OUTPUT/RESULT



THANK YOU